V-Lake: A Verifiable Lakehouse Architecture for Multi-Party Governed Data

Srinath Murali Rishav Kumar

Department of Computer Science
VIT University, Vellore, India

Abstract

Many regulated settings require several organisations to share structured data without any one of them holding unilateral authority over access, schema, or audit. Clinical trials, financial trade reporting, cross-institution scientific collaborations, and sustainability (ESG) disclosure are four examples. Existing data lakehouse platforms address storage and query federation but leave governance, integrity verification, and column-level privacy to external tooling.

We present **V-Lake**, an architecture for verifiable multi-party data governance over federated lakehouses. V-Lake is *not* a new cryptographic primitive. It is a specific composition of four mechanisms that, to our knowledge, have not previously been integrated at the lakehouse layer: (i) a domain-separated Merkle forest with on-chain root anchoring and a per-query chained compliance attestation; (ii) a *Weighted Quorum Governance* (WQG) layer—a voting scheme atop the blockchain's QBFT ordering, not a consensus algorithm—whose role weights are the unique minimal integer solution to six explicit constraints and whose power distribution we verify via Shapley–Shubik analysis; (iii) a DID-based consent chain providing subject-initiated delegation and revocation (a practical subset of W3C SSI/VC, not a full implementation); and (iv) a federated source registry spanning ten heterogeneous systems. On top of these, a hybrid column-level encryption scheme combines AES-SIV for near-unique identifiers with Fernet plus HMAC blind indexes over *deliberately coarsened* values for clinical attributes.

We use a multi-site clinical trial as the driving case study because it exercises all four pillars concurrently, but the mechanisms are domain-agnostic; §**??** sketches how the same design applies to trade reporting, ESG disclosure, and cross-institution research with only the sensitive-column registry and policy pack changing. We validate the design against a Hyperledger Besu (QBFT) network with a 830-line Solidity contract deployed and a Flask backend running in `mode=on-chain`, exercised through a benchmark artefact (`scripts/bench_full.py`) that hits the real contract. Measured mean end-to-end query overhead over raw DuckDB on five federated datasets is 10–21% (approximately 0.3–1.0 ms); Merkle build scales linearly from 0.06 ms at 10 rows to 232 ms at 50k rows; column-level encryption costs 0.03–0.08 ms per value; a real on-chain ingestion anchor round-trip (signed transaction plus block inclusion) takes a mean of 23 ms. Against a centralised IAM baseline (plain RBAC, no chain, no Merkle), V-Lake adds roughly 1 ms per query plus a one-time per-ingestion blockchain cost of tens of milliseconds. A test suite exercises seven modelled adversary classes; we do not claim formal security proofs or regulatory certification. We report the design, its rationale, its limitations, and the tradeoffs we consciously accept.

1 Introduction

Data lakehouses [**?**] consolidate the schema-on-read flexibility of data lakes with the transactional guarantees of data warehouses. In regulated multi-party settings, however, a lakehouse must additionally provide *data integrity*, *access governance*, *subject sovereignty*[1], and *privacy at rest*—properties that current architectures leave to external tooling.

We use a multi-site clinical trial as the driving case study because it exercises all four pillars concurrently: a pharmaceutical sponsor, a hospital site, and an ethics board share patient enrollment data (stored in S3), adverse-event reports (arriving via Kafka), laboratory results (in PostgreSQL), vitals telemetry (in MongoDB), and imaging documents (uploaded as PDFs). No single organisation should be able to unilaterally modify governance rules, grant access to sensitive columns, or alter data without detection. Patients enrolled in the trial should retain control over who sees their records, including the ability to delegate access to a physician and revoke it at will, without requiring approval from the governance layer.

The same four pillars appear with different names in other regulated domains—multi-bank trade reporting under MiFID II, corporate ESG disclosure under the EU CSRD, and cross-institution scientific data-use agreements among them. We return to this point in §**??**; the core architecture is not healthcare-specific.

1.1 What V-Lake is—and what it is not

V-Lake is a principled composition of four mechanisms, not a new cryptographic primitive. Each building block (Merkle trees, weighted voting games, decentralised identifiers, searchable encryption) is well-established. What is *new*, to our knowledge, is their specific integration at the lakehouse layer:

1. **Domain-Separated Merkle Forest (C1).** A typed hash tree with disjoint leaf/node domain tags and positional binding,

[1] Throughout, "subject" denotes the data principal—a patient in healthcare, a beneficial owner in finance, a disclosing entity in ESG—not a trial participant specifically.

extended with a cross-dataset forest root and a per-query hash-linked compliance attestation anchored on-chain.

2. **Weighted Quorum Governance (C2).** A governance voting scheme layered *above* the blockchain's QBFT ordering (it does not replace consensus). Role weights are derived as the unique minimal integer solution to six stated constraints (Proposition **??**); the resulting power distribution is cross-checked via Shapley–Shubik analysis.

3. **DID-Based Consent Chain (C3).** An ECDSA-signed, hash-linked consent log using decentralised identifiers as subject handles. It provides subject-initiated delegation and revocation but is a *subset* of the W3C SSI/VC model; we do not claim full VC-JSON interoperability (§**??**).

4. **Federated Data Source Registry (C4).** Ten connectors, each labelled as *materialising* the source into DuckDB or supporting predicate push-down (Table **??**).

On top of these, a hybrid column-level encryption scheme applies deterministic encryption for near-unique identifiers, randomised encryption for clinical values, and HMAC blind indexes over deliberately coarsened buckets when filterability is required.

The specific technical contributions are: (a) a *constraint-driven* derivation of role weights rather than parameters chosen by convention, together with a full Shapley–Shubik power-index check; (b) a cross-dataset Merkle forest whose root is tied, via a hash-linked attestation chain, to the *injected* SQL of each query (not merely to ingested rows); and (c) a per-column encryption mode assignment driven by column cardinality rather than applied uniformly.

We deliberately do not claim: formal security proofs, regulatory certification, resistance to adversaries outside the stated threat model, or full W3C VC interoperability. The evaluation is a prototype datapoint against a single multi-site trial scenario, not a broad performance claim.

The remainder of the paper proceeds as follows. Section **??** surveys prior art. Section **??** defines the system and threat models and provides a threat-mechanism matrix (Table **??**). Section **??** presents the architecture. Sections **??**–**??** detail each component. Section **??** covers access-control mechanics. Section **??** reports evaluation, methodology, and threats to validity. Section **??** sketches how the architecture applies beyond healthcare—to trade reporting, ESG disclosure, and cross-institution research—and what would change per domain. Section **??** discusses what is genuinely new versus inherited from prior work. Sections **??**–**??** conclude.

2 Prior Art

Data lakehouses. Delta Lake [**?**], Apache Iceberg [**?**], and Apache Hudi [**?**] provide ACID transactions and schema evolution over object-store data but delegate governance and access control to external catalogue services. V-Lake extends the lakehouse model with built-in verifiable governance rather than competing with these storage formats.

Blockchain-anchored data integrity. Anchoring Merkle roots on a blockchain is well-established in certificate transparency [**?**] and supply-chain provenance [**?**]. MedRec [**?**] applies Ethereum smart contracts to medical-record permission management. V-Lake builds on this insight but targets the lakehouse layer rather than EMR systems, and adds a cross-dataset forest root and per-query attestation chain absent in prior work.

Governance and weighted voting. Weighted voting has been studied extensively in cooperative game theory [**?**]. Systems such as Apache Ranger [**?**] and Open Policy Agent [**?**] enforce role-based policies but do not expose formal power-index analysis of their role hierarchies, and they execute unilaterally (a single administrator can change a policy). V-Lake's WQG layer differs on both points: weights are derived from explicit constraints, and critical operations require a multi-party quorum recorded on-chain.

Self-sovereign identity. Decentralised identifiers (DIDs) [**?**] and verifiable credentials [**?**] provide the foundation for subject-controlled access. V-Lake uses a DID-style subject handle and an ECDSA-signed consent chain but does not implement the full Verifiable Credentials data model or DIDComm messaging; we are explicit about this scoping in §**??**.

Searchable encryption. CryptDB [**?**] introduced onion encryption with deterministic and order-preserving layers for SQL. Subsequent work showed that order-preserving encryption leaks enough information to recover plaintexts on real medical data [**?**]. V-Lake avoids OPE entirely, using deterministic encryption only for near-unique identifiers and HMAC blind indexes with deliberate coarsening for clinical values.

3 System and Threat Model

3.1 Participants

V-Lake defines five roles with strictly ordered privilege levels: DATA STEWARD > DATA CUSTODIAN > ANALYST > SUBJECT > NONE. Multiple organisations each contribute stewards; no single organisation holds a majority.

3.2 Trust Assumptions

- The permissioned blockchain (Hyperledger Besu, QBFT) is trusted for ordering and availability but not for confidentiality.
- At least one steward is honest. As shown in §**??**, this suffices to block any critical governance change.
- The application server is trusted for computation but not for long-term storage: integrity is verified via Merkle proofs against on-chain roots.
- The query engine (DuckDB, in-memory) does not persist to disk, but we do not assume protection against OS-level memory attacks (swap, core dumps). Column-level encryption provides partial mitigation.

3.3 Threat Model

We consider seven adversary classes; Table **??** maps each class to the V-Lake mechanism that is *intended* to address it, and to the empirical evidence presented in §**??**. We stress that this is a design-and-test mapping, not a formal security proof.

Caveat on T5. The current row-filter validator is a deny-list, which is known to be weaker than a SQL-parser-based allow-list and may miss novel attack vectors. We treat this as a known limitation (§**??**), not a solved problem.

Caveat on T6. The column-encryption scheme has documented frequency-leakage tradeoffs on blind-indexed columns (§**??**). "Encrypted at rest" is not the same as "information-theoretically secure."

Adversaries *outside* these seven classes—for example, side-channel attacks on the host OS, a dishonest majority of stewards, or a compromise of the Besu network's validator set—are not addressed by this work.

4 Architecture Overview

Figure **??** illustrates V-Lake's pipeline; subsequent sections detail each component.

4.1 Pipeline

Data enters V-Lake through one of ten federated connectors (§**??**). Each source is either materialised into DuckDB's in-memory columnar engine or accessed via push-down (Table **??**). The pipeline then proceeds as follows:

1. **Column encryption.** PHI columns are identified by a default registry (overridable per dataset) and encrypted in place: deterministic encryption for identifiers, randomised encryption for clinical values, with blind-index side columns for coarsened equality filters (§**??**).
2. **Merkle commitment.** A domain-separated Merkle tree is built over the *post-encryption* rows, producing a root ρ that commits to the ciphertext state (§**??**).
3. **On-chain anchoring.** ρ is recorded on a Hyperledger Besu smart contract (QBFT, chain ID 1337, 2 s block period). A forest root Φ aggregates all dataset roots into a single verifiable hash.
4. **Governance.** Role assignments, policy attachments, and access grants are mediated by WQG (§**??**). Every governance action is recorded on-chain with a quorum certificate.
5. **Query processing.** Incoming queries pass through predicate injection (column and row restriction per the caller's grant), PHI predicate rewriting (translating plaintext literals to ciphertext or blind-index values), compliance checking (HIPAA, GDPR, DPDPA rules), and Merkle-proof generation. Results are decrypted for authorised callers only.

4.2 Technology Stack

The prototype is implemented as a Flask (Python) backend with DuckDB as the query engine, Hyperledger Besu as the permissioned blockchain (QBFT, 4 validators in reference config; a single-node dev instance in the benchmark), and MinIO, PostgreSQL, Kafka, and MongoDB as real federated sources. The smart contract (~830 LoC Solidity) mirrors the backend's governance state on-chain. The system is deployed via Docker Compose (8 containers). All measurements in §**??** are taken with the backend in `mode=on-chain`; measurements are invalidated if that flag is absent.

```
+-- Federated Sources (10 types) --+
| MinIO  Postgres  Kafka  Mongo .. |
+----------------+-----------------+
                 |
          DuckDB (in-memory)
                 |
        +--------+---------+
        | Column Encrypt.  |
        | det/rand/bidx    |
        +--------+---------+
                 |
        +--------+---------+
        | Merkle Forest C1 |
        | domain-separated |
        +--------+---------+
                 |
        +--------+---------+
        | Besu Contract    |
        | anchor + audit   |
        +--------+---------+
                 |
 +---------------+---------------+
 |               |               |
WQG (C2)   SSI Chain(C3)  Compliance
governance  consent log   HIPAA/GDPR
 |               |               |
 +---------------+---------------+
                 |
         Query Pipeline
      predicate injection
      PHI rewrite + decrypt
      Merkle proof bundle
                 |
              Caller
```

Figure 1: V-Lake architecture. Data flows top to bottom; governance and consent are cross-cutting concerns checked at query time.

5 C1: Domain-Separated Merkle Forest

5.1 Construction

We use SHA-256 with two domain-separated hash functions to prevent second-preimage confusion between leaves and internal nodes:

$$H_{\text{leaf}}(r, i) = \textsc{Sha256}\big(\texttt{"vlake.leaf:"} \,\|\, i \,\|\, \texttt{":"} \,\|\, n_c \,\|\, \texttt{":"} \,\|\, \hat{r}\big) \tag{1}$$

$$H_{\text{node}}(l, r, \ell) = \textsc{Sha256}\big(\texttt{"vlake.node:"} \,\|\, \ell \,\|\, \texttt{":"} \,\|\, l \,\|\, \texttt{":"} \,\|\, r\big) \tag{2}$$

Here i is the row's zero-based positional index, n_c is the column count, $\hat{r}$ is the canonical serialisation of the row, and ℓ is the tree level.

5.2 Design Properties

Three properties follow from this construction: *domain separation* ensures leaf and node hashes cannot collide under SHA-256 collision resistance; *position binding* (the index i appears in the leaf hash) prevents row-reorder attacks; *odd-leaf promotion* (unpaired nodes are promoted unchanged,

Table 1: Threat classes, V-Lake mechanisms intended to address them, and the evidence presented. "Test-validated" means a concrete adversarial scenario is implemented in our test suite and is observed to be rejected; it does not imply the mechanism is secure against all adversaries in the class.

#	Adversary class	Primary mechanism	Secondary	Evidence (§??)
T1	Data tampering (storage/transit)	Domain-separated Merkle forest (C1)	On-chain anchor	Test-validated: root mismatch
T2	Governance subversion (rogue coalition)	Weighted Quorum Governance (C2)	`all_stew` unanimity	Test-validated: 2/3 critical op denied
T3	Consent forgery (altered records)	SHA-256 hash-linked consent chain (C3)	ECDSA signatures	Test-validated: chain verify rejects
T4	Unauthorised column access	Non-Truman predicate injection (§**??**)	Grant allow-list	Test-validated: HTTP 403 returned
T5	SQL injection in row filter	Deny-list token validator	Parameter escaping	Test-validated; *see caveat*
T6	Stolen backup / rogue DBA (at rest)	Column-level encryption (§**??**)	Envelope key hierarchy	Ciphertext in DuckDB verified
T7	Expired or revoked grant reuse	Temporal check + grant cache invalidation	Redis-backed shared cache	Test-validated: rejection observed

not duplicated) prevents a length-extension variant where $\text{root}(A, B, C) = \text{root}(A, B, C, C)$. The empty sentinel root is $H_\emptyset = \text{SHA256}(\texttt{"vlake.empty_tree"})$.

5.3 Forest Root and On-Chain Anchoring

For each dataset j with Merkle root ρ_j, a forest leaf is $f_j = \text{SHA256}(\texttt{"vlake.forest:"}\|j\|\texttt{":"}\|\rho_j)$, and Φ is the Merkle root over the $\{f_j\}$. After every ingestion, V-Lake calls $\text{RECORDINGESTION}(\text{id}, \rho, |R|, |L|, \text{depth})$ on the Besu contract; the contract maintains a per-dataset `merkleHistory[id]` and the global Φ.

Each query additionally produces a chained attestation $a_t = \text{SHA256}(H(q_t)\|h_t\|\rho_t\|a_{t-1}\|\tau_t)$, where $H(q_t)$ is the *injected*-query hash (i.e., after predicate injection), h_t is the result hash, ρ_t the dataset root, a_{t-1} the previous attestation, and τ_t the timestamp. This chain, anchored on-chain, is the property that distinguishes V-Lake's integrity layer from a plain Merkle log: the SQL that produced each answer—not only the stored rows—is part of the tamper-evident record.

6 C2: Weighted Quorum Governance

6.1 Terminology: governance, not consensus

V-Lake's weighted voting layer is a *governance* scheme, not a *consensus* algorithm. Blockchain consensus (Besu's QBFT) orders transactions; WQG determines *which* governance transactions are ratified. Earlier drafts of this work used "Weighted Quorum Consensus" (WQC); we have retitled it to avoid confusion. Internal identifiers in our implementation retain the historical `WQC` prefix for backwards compatibility with the deployed ABI.

6.2 Weight Derivation

V-Lake assigns integer weights: Steward s=3, Custodian c=2, Analyst a=1. We state six requirements and prove that $(3, 2, 1)$ is the unique minimal integer solution.

R1 **Steward supremacy**: two stewards alone meet the standard quorum, i.e., $2s \geq q$.

R2 **No unilateral steward**: one steward alone cannot pass, i.e., $s < q$.

R3 **Custodian relevance**: a custodian is pivotal in at least one winning coalition.

R4 **Custodian insufficiency**: non-stewards alone cannot pass, i.e., $c + a < q$.

R5 **Strict ordering**: $0 < a < c < s$.

R6 **Integer weights**: $s, c, a \in \mathbb{Z}^+$ (for gas-efficient on-chain representation).

Fix the reference committee used throughout: three stewards, one custodian, one analyst, total $W = 3s + c + a$ and standard quorum $q = \lceil W/2 \rceil$.

Proposition 1. *Under **R1–R6** applied to the reference committee, $(s, c, a) = (3, 2, 1)$ is the unique minimal integer solution, where minimality is with respect to the lexicographic order (s, c, a).*

Proof. By **R5–R6**, $a \geq 1$, $c \geq a + 1 \geq 2$, and $s \geq c + 1 \geq 3$. We show $(3, 2, 1)$ satisfies all six constraints and that no lexicographically smaller integer triple does.

Feasibility of $(3, 2, 1)$. $W = 9 + 2 + 1 = 12$; $q = 6$. **R1**: $2s = 6 \geq 6$. ✓ **R2**: $s = 3 < 6$. ✓ **R4**: $c + a = 3 < 6$. ✓ **R3**: the coalition $\{S_1, S_2, C_1\}$ has weight $3 + 3 + 2 = 8 \geq 6$ (winning), but removing C_1 leaves $\{S_1, S_2\}$ with weight $6 \geq 6$ (still winning), so C_1 is *not* pivotal there. Consider instead $\{S_1, C_1, A_1\}$: weight $3 + 2 + 1 = 6 \geq 6$ (winning); $\{S_1, A_1\}$ has weight $4 < 6$ (losing), so C_1 is pivotal. ✓

Minimality. By **R5–R6** the lex-smallest candidate is $(s, c, a) = (3, 2, 1)$ itself; any strictly smaller triple must reduce s, c, or a, which violates **R5–R6**.

No feasible triple of equal s with smaller (c, a). Fix $s = 3$. Then $W = 9 + c + a$, $q = \lceil W/2 \rceil$. For $(c, a) = (2, 1)$ we showed **R1–R4** hold. Consider *any* (c', a') with $c' < 2$ or $a' < 1$: **R5** forces $c' \geq 2$ and $a' \geq 1$, so no strictly smaller (c, a) exists subject to **R5**.

Table 2: Shapley–Shubik power index for $[7; 3, 3, 3, 2, 2, 1]$.

Role	Weight	Pivots / 720	φ
Steward	3	156	0.217
Custodian	2	108	0.150
Analyst	1	36	0.050

Table 3: Quorum configuration per governance operation.

Operation	Type	Thr.	All S	C maj.
AssignCustodian	std	50%	–	–
OnboardAnalyst	std	50%	✓	✓
AccessGrant	std	50%	✓	✓
RevokeCustodian	std	50%	–	–
RevokeAnalyst	emerg	1 S	–	–
AttachPolicy	crit	67%	✓	–
ToggleConfidential	crit	67%	✓	✓

Combining, $(3, 2, 1)$ satisfies **R1–R6** and no lex-smaller integer triple does. Uniqueness of the minimal solution follows. □

Proposition 2 (Safety: no double ratification)**.** *For any quorum threshold $q \geq \lceil W/2 \rceil$, a proposal cannot be simultaneously approved (*Yes $\geq q$*) and rejected (*No $\geq q$*).*

Proof. Yes + No $\leq W$. If Yes $\geq q$, then No $\leq W - q \leq W - \lceil W/2 \rceil < q$ whenever $W \geq 2$, so No $< q$. □

Proposition 3 (Single-honest-steward sufficiency)**.** *For critical operations that require `all_stew`, a single honest steward suffices to block ratification regardless of the behaviour of all other participants.*

Proof. Ratification requires `stewardsMet` $\equiv$ every steward voted Yes. A single No from an honest steward falsifies this predicate. □

6.3 Shapley–Shubik Verification

To cross-check that the induced power distribution matches the intended hierarchy, we compute the Shapley–Shubik power index [**?**] for the reference game $[7;\ 3, 3, 3, 2, 2, 1]$ (three stewards, two custodians, one analyst) by full enumeration of all $6! = 720$ permutations (Table **??**). The ordering $\varphi(S) > \varphi(C) > \varphi(A)$ matches the weight ordering and the intended liability hierarchy.

6.4 Quorum Types

V-Lake defines three quorum types (Table **??**): *standard* (50% weighted threshold), *critical* (67% plus `all_stew`), and *emergency* (any single steward, for HIPAA §164.312 incident response).

7 C3: DID-Based Consent Chain

7.1 Scope and caveat

V-Lake's consent layer uses decentralised identifiers as subject handles and an ECDSA-signed, hash-linked log as the record of consent. This provides the *property* often associated with SSI—that a data subject can unilaterally delegate or revoke access without intermediation—but it is not a full W3C SSI/VC deployment. In particular:

- DIDs are derived locally as DID $=$ `"did:vlake:"` $\|$ $\text{SHA256}(\text{address}\|\tau)_{[0:16]}$; we do not register a DID method with the W3C.
- Consent records are JSON documents with a fixed schema; they are not VC-JSON or JWT-VC credentials.
- There is no DIDComm messaging layer; delegation is issued through the V-Lake API, not peer-to-peer.

Interoperability with other SSI stacks is future work (§**??**). The consent-chain design gives subjects the specific *agency* properties listed below, within the V-Lake ecosystem.

7.2 Consent Chain Structure

Each consent record c_i carries: `prev_hash` $= c_{i-1}$.`hash` (or `"genesis"` for c_0); `hash` $=$ SHA256(JSON(c_i)) with deterministic key ordering; `signature` $=$ ECDSA signature over the concatenation of action, dataset, filter, timestamp, and `prev_hash`. Chain verification asserts c_i.`prev_hash` $=$ c_{i-1}.`hash` at every step; any modification breaks all subsequent links.

7.3 Subject-Initiated Operations

Three operations require no governance-layer approval: *Link* (a steward binds a subject to a dataset with a SQL row filter); *Delegate* (a subject grants time-bounded access to a delegate, default 86 400 s); and *Revoke* (a subject unilaterally deactivates a delegate's grant and invalidates the query-engine cache immediately). This design gives patients agency over their records without requiring steward intervention for day-to-day delegation.

8 C4: Federated Data Source Registry

V-Lake supports ten source types (Table **??**). **Mode** indicates whether V-Lake materialises the source into DuckDB at ingestion time or issues pushed-down predicates at query time.

Table 4: Federated source types, connection method, and execution mode.

Source	Method	Mode
Local File (CSV/JSON)	DuckDB `read_csv_auto`	materialise
S3 / MinIO	DuckDB `httpfs`	materialise
PostgreSQL	DuckDB `postgres_scan`	push-down*
MySQL / MariaDB	DuckDB `mysql_scan`	push-down*
Trino / Starburst	Trino JDBC bridge	push-down
Delta Lake	DuckDB `delta_scan`	materialise
Apache Iceberg	DuckDB `iceberg_scan`	materialise
HDFS / Hadoop	`httpfs` + WebHDFS	materialise
Kafka	`kafka-python` consumer	materialise
MongoDB	`pymongo` export	materialise
Documents (PDF/DOCX/IMG)	Text extract + envelope enc.	materialise

* *Push-down caveat.* DuckDB's scan extensions push simple filter and projection predicates but materialise the remainder;

column-level encryption therefore runs on the materialised partition, not at the source. Sources labelled "push-down" are scalable beyond memory for filtered queries but degrade to "materialise" for unfiltered scans.

After materialisation (or push-down evaluation), every source goes through the same pipeline: column encryption, Merkle tree construction, and on-chain anchoring. Cross-source SQL joins are supported because all sources resolve to tables in the same DuckDB instance.

Documents follow an additional path: text is extracted (pypdf for PDF, XML parsing for DOCX), the raw binary is encrypted with the dataset's envelope key (Fernet: AES-128-CBC + HMAC-SHA256), and the ciphertext is persisted to a MinIO bucket. Plaintext bytes never survive beyond the upload handler.

The key design decision is *materialisation by default, with push-down where the source supports it*: a uniform point at which encryption, Merkle hashing, and access control can be applied consistently. The tradeoff is memory-bounded dataset size for materialised sources; we discuss this in §**??**.

9 Column-Level PHI Encryption

9.1 Motivation

HIPAA's Breach Safe Harbor provision applies only to encrypted PHI. V-Lake's document layer already encrypts file uploads at rest, but the structured-data path (DuckDB tables) previously stored rows in plaintext. We address this gap with a hybrid scheme that preserves SQL filterability where possible. We note explicitly that encryption at rest is a *necessary but not sufficient* condition for Safe Harbor eligibility; regulatory determination requires independent audit (§**??**).

9.2 Key Hierarchy

A master KEK wraps per-dataset DEKs. Three subkeys are derived per DEK via HKDF-like expansion:

$$k_{\text{siv}} = \textsc{Hkdf}\big(\text{DEK},\ \texttt{"vlake.colcrypt.siv.v1"},\ 64\big) \tag{3}$$

$$k_{\text{fernet}} = \textsc{Hkdf}\big(\text{DEK},\ \texttt{"vlake.colcrypt.fernet.v1"},\ 32\big) \tag{4}$$

$$k_{\text{bidx}} = \textsc{Hkdf}\big(\text{DEK},\ \texttt{"vlake.colcrypt.bidx.v1"},\ 32\big) \tag{5}$$

9.3 Three Encryption Modes

Deterministic (det). AES-SIV [**?**] with a 512-bit key. Identical plaintexts always produce identical ciphertexts, enabling equality filters and joins. Applied to *high-cardinality identifiers* (MRN, SSN, email, patient name) where frequency leakage is minimal.

Randomised (rand). Fernet (AES-128-CBC + HMAC-SHA256) with a fresh IV per call. Applied to *clinical values* (diagnosis, DOB, ZIP, notes).

Blind index (bidx). Truncated HMAC-SHA256 over a *coarsened* value, stored in a side column (`_bidx_`*`col`*). Enables equality filters with bounded, documented frequency leakage. Three coarseners are provided: `year` (DOB → year), `zip3` (first 3 ZIP chars), `icd3` (first 3 ICD-10 chars, e.g., E11.9 → E11).

Table 5: Column encryption mode assignment by column type.

Column Type	Mode	Blind Idx	Rationale
MRN, SSN, email, name	det	–	Near-unique; flat freq.
DOB, birth date	rand	year	Quasi-identifier
ZIP, postal code	rand	zip3	Quasi-identifier
Diagnosis, ICD code	rand	icd3	Disease-family bucket
Notes, free text	rand	–	No filtering needed
Non-PHI columns	–	–	Plaintext

A column that is `rand`-encrypted *and* has a blind index gets random ciphertext on the stored value while the side column enables coarse equality filters. Table **??** summarises the assignment.

9.4 Query Rewriting

Equality predicates on PHI columns are rewritten before reaching DuckDB. `det` columns: the literal is replaced with its deterministic ciphertext. `rand+bidx` columns: the predicate targets the blind-index side column with the HMAC of the coarsened value. `rand`-only columns: the predicate becomes `FALSE` (we prefer an explicit zero-row result to a silent match of every row). Range queries, `LIKE`, and function calls on encrypted columns are not rewritten.

9.5 Frequency-Leakage Budget

We are explicit about what this scheme leaks. Deterministic encryption on near-unique columns produces a flat frequency distribution. Blind indexes produce a bucketed distribution; an attacker can correlate bucket frequencies with published prevalence data (the classic [**?**] failure mode), which is why the coarsening is chosen per column (family-level for ICD) rather than left at the raw value. `rand`-only columns leak no frequency information. Blind-index keys are per-dataset, so compromising one dataset does not break frequency analysis across the lake. This is a *documented tradeoff*, not a solved problem.

10 Access Control and Compliance

10.1 Non-Truman Predicate Injection

When a grant restricts a caller to a subset of columns or rows:

- **Column restriction**: `SELECT *` is rewritten to an explicit column list. Queries naming unauthorised columns return HTTP 403 with the disallowed column list—the "Non-Truman" model [**?**], which rejects rather than silently filters.
- **Row restriction**: the grant's SQL predicate is prepended as `WHERE (`*`filter`*`) AND (`*`original`*`)`.

10.2 SQL Injection Prevention (known-weak)

User-supplied row filters are validated against a deny-list of dangerous tokens. This is a *known weaker* defence than a SQL-parser-based allow-list and may miss novel attack forms. A

production deployment should integrate a parser; we flag this explicitly in Table **??** (T5) and §**??**.

10.3 Compliance Engine

Policy rules are grouped into *policy packs* that define the set of sensitive columns and the rules to apply. The current prototype ships healthcare-oriented packs (HIPAA, GDPR, India DPDPA) but the pack format is a plain declarative registry; finance (SOX, MiFID II, PCI-DSS), disclosure (EU CSRD), and payments (PCI-DSS) packs are straightforward additions that do not change the engine. The engine checks four properties per query: column exposure (informational), temporal validity, audit logging, and minimum-necessary (which blocks `SELECT *` on datasets with sensitive columns). Each policy emits a chained attestation whose hash feeds the next policy's input; the overall chain is anchored on-chain. The word "compliance" in this section refers to rule checks matching the pack's patterns, not to regulatory certification.

10.4 Grant Lifecycle

Grants are created through WQG proposals; they carry an access level (VIEWONLY, VIEWDOWNLOAD, FULLACCESS), optional column restriction, optional row filter, and expiration. A TTL-based grant cache (30 s, Redis-backed in multi-instance deployments) avoids repeated policy lookups; revocation invalidates the cache immediately.

11 Evaluation

11.1 Methodology

All measurements are taken against the real stack: backend running with `mode=on-chain`, the 830-line Solidity contract deployed to a Hyperledger Besu (QBFT, dev-mode) node, and the same five federated datasets seeded by the demo walkthrough. The benchmark driver is `scripts/bench_full.py`; it is reproducible and is the artefact behind every number in this section. The driver aborts if the backend is not in on-chain mode, preventing accidental numbers from a simulated or cache-only run.

For each operation we report n, mean, standard deviation, 95% confidence interval (normal approximation), and the p50/p95/p99 percentiles. Query microbenchmarks use $n = 200$ after 5 warmup iterations; Merkle benchmarks use n=5 per size; on-chain round-trips use n=10 for anchor ingestion and n=5 for the full propose-and-vote cycle owing to the much higher per-sample cost. The measurement host is a Windows 11 laptop with the Besu, backend, and datastore containers co-located via Docker Compose over an internal bridge network; RPC calls therefore include only localhost and intra-container latency. Real-network latency will be strictly higher.

11.2 Datasets and Representativeness

Five datasets from the demo walkthrough are loaded from four distinct backends (MinIO, PostgreSQL, MongoDB, local documents) with row counts of 8, 0, 15, 30, and 1. These sizes are representative of a *per-site* clinical-trial extract at the prototype stage; they are not a scale benchmark, and we report Merkle scaling separately to address that axis. We do not claim generalisation to production-scale data warehouses.

Table 6: End-to-end query overhead: V-Lake (full pipeline) vs raw DuckDB, per dataset. n=200 iterations after warmup. Measurements from `scripts/bench_full.py` on the running on-chain stack.

Dataset	**Baseline (ms)**		**V-Lake (ms)**	
	avg	p95	avg	p95
trial_enrollment	4.04	6.33	4.88	7.57
lab_results	3.03	4.36	3.33	4.49
vitals_stream	4.92	8.11	5.60	7.63
imaging_reports	3.87	6.81	4.44	6.96
Mean	3.97	6.40	4.56	6.66
Overhead: +0.59 ms avg, +0.26 ms p95 ($\approx$ 10–21%)				

Table 7: Centralised IAM baseline vs V-Lake increments. The centralised baseline uses a synthetic table and an in-memory ACL, so it is an *upper bound* on what a conventional stack achieves.

Operation	**Centralised**	**V-Lake**
Query path (mean)	1.94 ms	4.56 ms
Query path (p95)	3.01 ms	6.66 ms
Admin grant (single writer)	$\sim$ 0 ms	n/a
WQG proposal + 2 on-chain votes (mean)	n/a	18.84 ms
Ingestion Merkle anchor (on-chain)	n/a	23.37 ms
Tamper detection	none	root mismatch
Multi-party separation of duties	none	WQG
Subject-initiated revocation	none	C3
Cross-instance audit integrity	none	on-chain

11.3 End-to-End Query Overhead

Table **??** reports mean, p95, and relative overhead of the full V-Lake pipeline over raw DuckDB for each dataset. The V-Lake path includes predicate injection, compliance checking, PHI predicate rewriting, Merkle-proof generation, and attestation hashing; the baseline path is DuckDB alone.

11.4 Comparison to a Centralised IAM Baseline

A natural question is what V-Lake *costs* relative to a conventional centralised stack. We report a best-case centralised baseline: a Python dict-lookup RBAC check plus a raw DuckDB query on a synthetic 1,000-row table, with no chain, no Merkle tree, no compliance attestation, and an append-only in-memory audit list (Table **??**). A single-admin grant mutation is essentially a dict write, measured at $\sim$ 0 ms.

Takeaway. Per query, V-Lake adds approximately 1–3 ms over a centralised RBAC path—a constant-factor cost dominated by Merkle-proof generation and compliance hashing. The large costs (tens of milliseconds) are *per-ingestion* and *per-governance-action*, not per-query. In exchange, V-Lake provides four properties that the centralised baseline has no way to offer: integrity verification, separation of duties, subject sovereignty, and cross-instance audit integrity. Whether that cost is worth paying is a deployment decision; our claim is only that the incremental cost is small and the extra properties are

Table 8: Merkle forest build time vs row count (n=5 per size).

Rows	Avg (ms)	P95 (ms)
10	0.06	0.07
100	0.34	0.37
1 000	3.56	3.83
5 000	19.15	22.82
10 000	36.01	38.44
50 000	231.87	279.69

Table 9: Column encryption kernel microbenchmark.

Operation	Mean (ms)	P95 (ms)
Deterministic (AES-SIV) encrypt+decrypt	0.03	0.05
Randomised (Fernet) encrypt+decrypt	0.08	0.12
Blind-index HMAC (icd3 coarsener)	0.00	0.01

measurable.

11.5 Merkle Build Scaling

Merkle construction is $O(n)$; Table **??** shows the measured numbers match this shape. The 50k-row case builds in 232 ms mean.

11.6 Column Encryption Kernel

The column-encryption primitives are fast enough that their cost is dominated by Python overhead; they are not a bottleneck in the query path (Table **??**).

11.7 On-Chain Round-Trip

Every anchor and every governance action is a real signed transaction submitted to Besu; the benchmark waits for the transaction receipt before recording the sample (Table **??**). Anchor ingestion (one TX) has a mean round-trip of 23.37 ms; a full proposal-plus-two-vote cycle (three TXs) has a mean round-trip of 18.84 ms. The two numbers are close because Besu in dev-mode produces blocks on demand when the transaction pool is non-empty rather than waiting strictly for the 2 s period; the QBFT configuration would dominate on a production network with validators separated by a real network.

These are not simulated values. They are taken with the backend calling the contract deployed at `0xb9A2...C0cc` on the running Besu node; removing the Besu container causes the backend health endpoint to switch out of on-chain mode and the benchmark to abort.

11.8 Adversary Scenarios (Scoped)

We implement and run seven concrete adversary scenarios (Table **??**). Each scenario is a deterministic unit test: the benchmark calls the endpoint, observes whether the adversary succeeds, and reports the outcome. Every scenario is rejected by the tested defence in this configuration. We are careful to say "test-validated" rather than "provably secure": these tests exhaust neither the stated threat class nor the adversary space outside the threat model. The row-filter validator in particular is a deny-list and is flagged as a known-weak defence (T5).

Table 10: Real on-chain round-trip (Besu QBFT, dev-mode, localhost). Each sample includes signing, submission, and receipt wait.

Operation	n	Mean (ms)	P95 (ms)
Merkle-root anchor (1 TX)	10	23.37	32.00
WQG proposal + 2 steward votes (3 TX)	5	18.84	31.90

Table 11: Adversary scenarios and the defence that rejected them in our test suite. "Test-validated" is not a security proof.

Scenario	Rejected	Defence
SQL injection in row filter	✓	Deny-list (weak; see §**??**)
Unauthorised column access	✓	Non-Truman rejection
Role escalation (analyst votes)	✓	Role check at vote time
Expired grant reuse	✓	Temporal check in query engine
Data tampering	✓	Merkle root mismatch
Consent chain forgery	✓	Hash-chain verification
2/3 stewards on critical op	✓	`all_stew` unanimity

11.9 Threats to Validity

External validity. The evaluation is a single clinical-trial scenario with small per-dataset row counts. We present Merkle scaling, crypto microbench, and on-chain round-trip separately to give axis-specific evidence, but the end-to-end latency numbers should be read as *prototype datapoints*, not production performance claims.

Measurement noise. On datasets of 8–30 rows the raw DuckDB query itself takes 3–5 ms, so the V-Lake overhead (0.3–1.0 ms on average) is close to the measurement noise floor. We report confidence intervals and percentile spreads so readers can assess this directly; larger-row studies are a natural follow-up.

Internal validity. Benchmarks run co-located with the backend on localhost; real-network latency, contention from other workloads, and Besu validator-to-validator propagation on a production network will increase the numbers, particularly for the on-chain round-trip. We report the dev-mode Besu figure honestly but note that a 2 s QBFT block period on a real network would dominate those measurements.

12 Discussion: What Is Genuinely New

It is worth being explicit about what in this work is new versus standard composition of known primitives, and what is not. We split the answer into three categories.

Inherited, unchanged. SHA-256 Merkle trees, ECDSA signatures, AES-SIV, HMAC-SHA256, QBFT consensus, the Shapley–Shubik power index, the Non-Truman access-control model, and the observation that order-preserving encryption leaks too much on medical data. None of these are contributions.

Adapted, with small modifications. The domain-separated Merkle hashing style is folklore from transparency logs; our

contribution here is consistent application (including position binding and odd-leaf promotion) and the aggregation of per-dataset roots into a forest root with its own domain tag. The Fernet-plus-blind-index pattern exists in various SDKs; our contribution is the *cardinality-driven* mode assignment and explicit coarsening budget.

New, to our knowledge, at the lakehouse layer. (i) A constraint-driven derivation of weighted-voting weights (Proposition **??**) rather than parameters chosen by convention, paired with a full power-index check—most RBAC/policy stacks use convention and do not publish either. (ii) A per-query chained attestation that binds the *post-injection* SQL to the Merkle root and anchors the chain on-chain; this makes query-level tamper detection possible, not just row-level. (iii) The specific composition of a cross-dataset forest root, WQG governance, a DID-style consent chain, and hybrid column encryption behind a single query pipeline.

What is explicitly *not* new and we do not claim otherwise: no new cryptographic primitive; no formal security proof; no regulatory certification; no full W3C SSI/VC implementation; and no generalisation of the evaluation beyond the prototype configuration described.

13 Limitations and Future Work

1. **In-memory materialisation.** DuckDB runs in-memory, limiting materialised dataset size to RAM. Push-down sources (Postgres, MySQL, Trino) scale beyond this but degrade for unfiltered scans. Production deployments would benefit from spill-to-disk with encrypted swap or a persistent encrypted engine.

2. **Blind-index frequency leakage.** Coarsening reduces leaked information but does not eliminate it. Blind indexes on low-cardinality columns still reveal bucketed frequency distributions. Documented tradeoff, not a solved problem.

3. **Predicate rewriter scope.** The current rewriter supports only simple equality predicates on PHI columns. Range queries, `LIKE`, and functions on encrypted columns are not rewritten. Range queries would require order-revealing encryption, which we deliberately avoid [**?**].

4. **Row-filter validation is deny-list based.** The deny-list is known weaker than a parser-based allow-list. A production deployment should integrate a SQL parser. This is flagged in Table **??** (T5).

5. **DID/SSI interoperability.** The consent chain is not a full W3C VC/DIDComm implementation (§**??**). Interop with external SSI stacks is future work.

6. **No formal security proof.** Security claims are supported by test-validated scenarios (§**??**) rather than formal proofs. Symbolic or game-based analysis is future work.

7. **No regulatory certification.** Phrases such as "HIPAA Safe Harbor" and "GDPR Art. 32" refer to design alignment, not to third-party certification. Real deployment requires independent audit.

8. **Benchmark scope.** Numbers in §**??** reflect a single host, dev-mode Besu, and small datasets. Scaling to larger committees, larger policy sets, and real-network validators is future work.

14 Conclusion

V-Lake demonstrates that verifiable governance, tamper-evident integrity, subject-initiated delegation, and privacy-preserving column encryption can be composed into a lakehouse architecture with small per-query overhead (approximately 1 ms over a centralised IAM baseline on our prototype) and tens of milliseconds per ingestion anchor on a real blockchain. The contribution is not a new primitive but a specific composition: weights derived from explicit constraints and verified via Shapley–Shubik, a Merkle forest whose root is tied to per-query attestations, a DID-style consent chain scoped to subject-initiated delegation, and a column-encryption scheme with a documented frequency-leakage budget.

We have been deliberate about what the paper does and does not claim. It does not claim a new cryptographic primitive, a formal security proof, regulatory certification, or generalisation beyond the prototype. It does claim that the composition is coherent, the weight derivation is minimal and unique under the stated constraints, and the measured overhead is small enough to be a practical alternative to centralised IAM when the additional properties (multi-party separation of duties, tamper-evident integrity, subject sovereignty, cross-instance audit) are required.

Reproducibility. All numbers in this paper are produced by `scripts/bench_full.py` against the running stack with the backend in `mode=on-chain`. The script aborts if the stack is not live; removing the Besu container breaks it. This is deliberate: we do not want the paper's numbers to be mistakable for simulation output.

References

[1] M. Armbrust, A. Ghodsi, R. Xin, and M. Zaharia, "Lakehouse: A New Generation of Open Platforms that Unify Data Warehousing and Advanced Analytics," in *Proc. CIDR*, 2021.

[2] M. Armbrust *et al.*, "Delta Lake: High-Performance ACID Table Storage over Cloud Object Stores," *Proc. VLDB Endow.*, vol. 13, no. 12, pp. 3411–3424, 2020.

[3] R. Blue, O. Meara, and D. McIntosh, "Apache Iceberg: An Open Table Format for Huge Analytic Datasets," 2020. https://iceberg.apache.org

[4] V. Jain *et al.*, "Apache Hudi: The Streaming Data Lake Platform," 2020. https://hudi.apache.org

[5] B. Laurie, A. Langley, and E. Käsper, "Certificate Transparency," RFC 6962, IETF, 2013.

[6] K. Toyoda, P. T. Mathiopoulos, I. Sasase, and T. Ohtsuki, "A Novel Blockchain-Based Product Ownership Management System (POMS) for Anti-Counterfeits in the Post Supply Chain," *IEEE Access*, vol. 5, pp. 17465–17477, 2017.

[7] A. Azaria, A. Ekblaw, T. Vieira, and A. Lippman, "MedRec: Using Blockchain for Medical Records, Decentralized Content Management," in *Proc. IEEE Int. Conf. Open and Big Data*, 2016, pp. 25–30.

[8] L. S. Shapley and M. Shubik, "A Method for Evaluating the Distribution of Power in a Committee System," *Amer. Polit. Sci. Rev.*, vol. 48, no. 3, pp. 787–792, 1954.

[9] Apache Software Foundation, "Apache Ranger," 2023. https://ranger.apache.org

[10] Open Policy Agent, "OPA: Policy-based control for cloud native environments," 2023. https://www.openpolicyagent.org

[11] W3C, "Decentralized Identifiers (DIDs) v1.0," W3C Recommendation, 2022.

[12] W3C, "Verifiable Credentials Data Model v2.0," W3C Recommendation, 2024.

[13] R. A. Popa, C. M. S. Redfield, N. Zeldovich, and H. Balakrishnan, "CryptDB: Protecting Confidentiality with Encrypted Query Processing," in *Proc. SOSP*, 2011, pp. 85–100.

[14] M. Naveed, S. Kamara, and C. V. Wright, "Inference Attacks on Property-Preserving Encrypted Databases," in *Proc. CCS*, 2015, pp. 644–655.

[15] P. Rogaway and T. Shrimpton, "Deterministic Authenticated-Encryption: A Provable-Security Treatment of the Key-Wrap Problem," in *Proc. EUROCRYPT*, 2006, pp. 373–390.

[16] S. Rizvi, A. Mendelzon, S. Sudarshan, and P. Roy, "Extending Query Rewriting Techniques for Fine-Grained Access Control," in *Proc. SIGMOD*, 2004, pp. 551–562.